

A Project Report On

StudyStore

Submitted in partial fulfillment of the requirement for the
award of the degree

Master of Computer Applications
(MCA)

Academic Year 2026 – 27

Kashyap Bharad (92500584001)

Dhaval Parmar (92500584066)

Chirag Pandya (92500584070)

Internal Guide

Dr. Jaypalsinh Gohil



Marwadi
University
Marwadi Chandarana Group





Marwadi
University
Marwadi Chandarana Group



Faculty of Computer Applications (FoCA)

Certificate

**This is to certify that the project work entitled
StudyStore**

**submitted in partial fulfillment of the requirement for
the award of the degree of
Master of Computer Applications (MCA)
of the**

Marwadi University

is a result of the bonafide work carried out by

Kashyap Bharad (92500584001)

Dhaval Parmar (92500584066)

Chirag Pandya (92500584070)

during the academic year 2026-27

Faculty Guide

HOD

Dean

DECLARATION

We hereby declare that this project work entitled StudyStore – Online Stationery Shopping System is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Place: Rajkot

Date:

Kashyap Bharad (92500584001)

Dhaval Parmar (92500584066)

Chirag Pandya (92500584070)

Signature:_____

Signature:_____

Signature:_____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

We are very thankful to our guide **Dr. Jaypalsinh Gohil** the person who makes us to follow the right steps during our project work. We express our deep sense of gratitude to for his /her guidance, suggestions and expertise at every stage. A part from that his/her valuable and expertise suggestion during documentation of our report indeed help us a lot.

Thanks to our friend and colleague who have been a source of inspiration and motivation that helped to us during our project work.

We are heartily thankful to the Dean of our department **Dr. Rijwan Khan** sir and HoD **Dr. Sunil Bajaja** sir for giving us an opportunity to work over this project and for their end-less and great support to all other people who directly or indirectly supported and help us to fulfil our task.

Kashyap Bharad (92500584001)
Dhaval Parmar (92500584066)
Chirag Pandya (92500584070)

Signature:_____
Signature:_____
Signature:_____

CONTENTS

Chapters	Particulars	Page No.
1	Introduction to Project Definition	1
2	PREAMBLE	3
2.1	Module description	
3	REVIEW OF LITERATURE	5
4	TECHNICAL DESCRIPTION	7
4.1	Hardware Requirement	
4.2	Software Requirement	
5	SYSTEM DESIGN AND DEVELOPMENT	9
	• Diagrams as applicable • - Algorithm • - Flow Chart • - Data Flow Diagram • - Class Diagram • - Use Case Diagram • - Sequential Diagram • - Activity Diagram • - State Diagram • Database Design / File Structure (If applicable) • Screen Design • Code of the module	
6	SYSTEM TESTING	44
7	CONCLUSION	46
8	LEARNING DURING PROJECT WORK	47
8.1	Future Enhancement	
9	BIBLIOGRAPHY	48
9.1	Online References	
9.2	Offline References	

Table Index

Table No.	Table Title	Page No.
Table 4.1	Hardware Requirements	7
Table 4.2	Software Requirements	7
Table 5.1	Users Collection Structure	21
Table 5.2	Wishlist Collection Structure	21
Table 5.3	Products Collection Structure	22
Table 5.4	Cart Collection Structure	22
Table 5.5	Orders Collection Structure	23
Table 5.6	Collection Relationships	23

Figure Index

Figure No.	Figure Title	Page No.
Figure 5.1	Customer Flowchart	9
Figure 5.2	Administrator Flowchart	11
Figure 5.3	Context Diagram	12
Figure 5.4	Level 1 Data Flow Diagram	13
Figure 5.5	Class Diagram	14
Figure 5.6	Use Case Diagram	15
Figure 5.7	Sequence Diagram	16
Figure 5.8	Customer Activity Diagram	18
Figure 5.9	Administrator Activity Diagram	20
Figure 5.10	Home Screen	24
Figure 5.11	Registration Screen	24
Figure 5.12	Login Screen	25
Figure 5.13	Product Listing Screen	25
Figure 5.14	Product Details Screen	26
Figure 5.15	Wishlist Screen	26
Figure 5.16	Shopping Cart Screen	27
Figure 5.17	User Management Screen	27
Figure 5.18	My Orders Screen	28
Figure 5.19	Profile Screen	28
Figure 5.20	Forgot Password Screen	29
Figure 5.21	Admin Dashboard Screen	29
Figure 5.22	Product Management Screen	30
Figure 5.23	Order Management Screen	30
Figure 5.24	Coupon Management Screen	31
Figure 5.25	Checkout Screen	31
Figure 5.26	Razorpay UPI Payment Screen	32

➤ SYNOPSIS

• Introduction

StudyStore – Online Stationery Shopping System is a full-stack web application developed to provide a secure and user-friendly platform for purchasing stationery products online. The system allows customers to browse, search, and purchase stationery products, while administrators can efficiently manage products and customer orders. It is developed using the **MERN Stack**, with **React.js (Vite)** for the frontend, **Node.js** and **Express.js** for the backend, and **MongoDB Atlas** for database management. **JWT** is used for secure user authentication, **bcryptjs** for password encryption, and **Nodemailer** for email-based password recovery.

• Project Motivation

Although many e-commerce websites offer stationery products, they mainly focus on multiple product categories. StudyStore is designed as a dedicated online stationery shopping platform that provides a simple, secure, and responsive shopping experience. The project also demonstrates the practical implementation of modern web development technologies and software engineering concepts learned during the MCA program.

• Objectives.

- To provide a simple and user-friendly shopping experience.
- To implement secure user authentication using JWT.
- To provide product search and category filtering.
- To implement wishlist and shopping cart functionality.
- To simplify the online ordering process.
- To provide an admin dashboard for managing products and orders.

• 4. Key Features

- User Registration, Login, and Secure JWT Authentication
- Forgot Password and Reset Password
- Product Search, Category Filter, and Product Browsing
- Wishlist and Shopping Cart Management
- Order Placement and Order History
- Admin Dashboard for Product and Order Management
- Responsive and User-Friendly Interface

➤ Expected Outcome

The StudyStore application provides a convenient platform for purchasing stationery products online while enabling administrators to manage products and customer orders efficiently. The project demonstrates the practical implementation of full-stack web development, secure authentication, database management, and RESTful APIs in a real-world e-commerce application.

Chapter 1: INTRODUCTION TO PROJECT DEFINITION

1.1 Introduction

StudyStore – Online Stationery Shopping System is a full-stack web application developed to provide an online platform for purchasing stationery products. The application allows customers to browse products, search items, add products to their wishlist or shopping cart, and place orders through a simple web interface. It also provides an administrator panel for managing products and customer orders.

The application is developed using the MERN Stack, where React.js (Vite) is used for the frontend, Node.js and Express.js for the backend, and MongoDB Atlas for database management. JWT is used for user authentication, bcryptjs for password encryption, and Nodemailer for email-based password recovery.

1.2 Project Definition

StudyStore is a web-based stationery shopping system designed to simplify the online purchase of stationery products. The application provides features such as user registration, login, product browsing, product search, wishlist management, shopping cart, order placement, and order tracking. It also includes an administrator dashboard for managing products and customer orders.

1.3 Problem Statement

Many online shopping websites sell stationery products along with several other product categories, making it difficult for users to quickly find the required items. There is a need for a dedicated stationery shopping platform that focuses on stationery products while allowing customers to browse, search, and place orders. The system also provides administrators with a centralized platform to manage products and customer orders.

1.4 Objectives of the Project

- To develop an online stationery shopping system.
- To provide product browsing, searching, and category filtering.
- To implement user registration and login using JWT.
- To provide wishlist and shopping cart functionality.
- To support online order placement and order tracking.
- To provide an administrator dashboard for managing products and orders.

1.5 Scope of the Project

The StudyStore application provides an online platform for purchasing stationery products. Customers can register, log in, browse products, search items, manage their wishlist and shopping cart, place orders, and view their order history. Administrators can manage products

and customer orders through a dedicated dashboard. The project demonstrates the practical implementation of full-stack web development using the MERN Stack.

1.6 Key Features

- User Registration and Login
- Product Browsing, Search, and Category Filter
- Wishlist and Shopping Cart Management
- Order Placement and Order History
- Password Recovery through Email
- Admin Dashboard for Product and Order Management
- Responsive User Interface

1.7 Expected Outcome

The StudyStore application provides an online platform for purchasing stationery products and managing products and customer orders. The project demonstrates the practical implementation of a full-stack web application using React.js, Node.js, Express.js, MongoDB Atlas, JWT authentication, and RESTful APIs.

Chapter 2 : PREAMBLE

➤ Module Description

The StudyStore application is divided into several functional modules, where each module performs a specific task to provide a complete online stationery shopping experience. The details of each module are described below.

2.1 User Authentication Module

The User Authentication Module manages user registration, login, and password recovery. It accepts user details such as name, email address, password, and login credentials as input. During processing, the system validates the entered information, encrypts passwords using **bcryptjs**, authenticates users using **JWT**, and sends password reset links through **Nodemailer** when required. As a result, users can successfully register, log in, recover forgotten passwords, and access protected features of the StudyStore application.

2.2 User Profile Module

The User Profile Module allows users to view and update their personal information. It accepts profile details such as name, email address, mobile number, and other account information as input. The system retrieves the existing information from the database, validates the updated details, and stores the modified information. As a result, users can manage their profiles and keep their account information up to date.

2.3 Product Management Module

The Product Management Module manages all stationery products available in the application. It accepts product details such as product name, category, price, description, product image, and stock information as input. The system stores, retrieves, updates, and deletes product information in the MongoDB database according to administrator actions. As a result, customers can browse available products, while administrators can efficiently manage the product catalog.

2.4 Search and Category Filter Module

The Search and Category Filter Module helps users quickly locate stationery products. It accepts search keywords and selected product categories as input. The system searches the database for matching products, filters the results according to the selected category, and displays the relevant products. As a result, users can easily find the required stationery items.

2.1 Wishlist Module

The Wishlist Module enables users to save products for future purchase. It accepts the selected product and user account information as input. The system stores wishlist items in the database, retrieves saved products, and allows users to remove products or move them to the shopping cart. As a result, users can maintain a personalized list of products for future reference.

2.6 Shopping Cart Module

The Shopping Cart Module allows users to manage products before placing an order. It accepts selected products, quantities, and user account information as input. The system adds products to the cart, updates quantities, removes unwanted items, calculates the total amount, and prepares the order for checkout. As a result, users can review their selected products and complete the purchasing process.

2.7 Order Management Module

The Order Management Module manages the complete order process from order placement to order tracking. It accepts customer details, shipping information, cart items, and order details as input. The system stores order information in the database, displays order history to customers, and allows administrators to update the order status. As a result, customers can place orders, track their status, and view their order history.

2.8 Admin Dashboard Module

The Admin Dashboard Module provides administrators with a centralized interface to manage the application. It accepts administrator credentials, product information, and customer order details as input. The system authenticates the administrator and provides options to add, update, or delete products and monitor customer orders. As a result, administrators can efficiently manage the overall operation of the StudyStore application.

2.9 Password Recovery Module

The Password Recovery Module allows users to reset forgotten passwords. It accepts the registered email address and the new password as input. The system verifies the user account, sends a password reset link using **Nodemailer**, encrypts the new password using **bcryptjs**, and updates it in the database. As a result, users can successfully recover their accounts and continue using the application.

Chapter 3: REVIEW OF LITERATURE

3.1 Introduction

A literature review is an important phase of software development that helps in understanding existing systems, studying their features, identifying their limitations, and selecting suitable technologies for the proposed solution. For the StudyStore project, several online shopping platforms and stationery-focused websites were analyzed to understand current trends in e-commerce applications and identify opportunities for developing a dedicated online stationery shopping system.

3.2 Study of Existing Systems

Various online shopping platforms such as **Amazon, Flipkart, and Meesho** were studied during the development of this project. These platforms provide stationery products along with features like user registration, product search, wishlist, shopping cart, order placement, and order tracking. Dedicated stationery websites such as **Scooboo, Odd Giraffe, and Hindustan Stationery** were also analyzed to understand product organization and category-based shopping.

Major Observations

- General e-commerce websites offer stationery products along with many other product categories.
- Dedicated stationery websites provide a more organized shopping experience.
- Most systems include user registration, product search, wishlist, shopping cart, and order management.
- A responsive and user-friendly interface improves the overall shopping experience.

3.3 Comparative Analysis

The study of existing systems shows that general e-commerce platforms are designed to sell products from multiple categories, while dedicated stationery websites focus only on stationery items. Based on this analysis, **StudyStore** is developed as a dedicated online stationery shopping application that provides essential e-commerce features in a simple and organized manner. The application integrates customer services and administrator functions into a single platform using the MERN Stack.

Highlights of StudyStore

- Dedicated online stationery shopping platform.
- Simple and responsive user interface.
- Product search and category filtering.
- Wishlist and shopping cart management.
- Order placement and order tracking.
- Admin dashboard for product and order management.
- Developed using the MERN Stack.

3.4 Limitations of Existing Systems

Based on the study of existing applications, the following limitations were identified:

- Stationery products are often mixed with many unrelated product categories.
- Large e-commerce platforms contain features that are not necessary for a dedicated stationery shopping system.
- Product navigation may become complex due to the large number of available categories.
- Some stationery websites focus mainly on product sales and provide different levels of administrative functionality.

These observations helped in identifying the requirements for the proposed StudyStore application.

3.5 Proposed System

StudyStore is proposed as a dedicated online stationery shopping system developed using the MERN Stack. The application allows customers to register, browse stationery products, search items, manage their wishlist and shopping cart, place orders, and view order history. It also provides administrators with a centralized dashboard for managing products and customer orders.

Advantages of the Proposed System

- Dedicated platform for stationery shopping.
- Organized product categories.
- Easy product search and browsing.
- Integrated wishlist and shopping cart.
- Simple order management process.
- Centralized administrator dashboard.
- Modular architecture for future enhancements.

3.6 Summary

The literature review provided a clear understanding of existing online shopping platforms and stationery websites. Their features, strengths, and limitations were carefully analyzed to identify the requirements of the proposed system. Based on this study, **StudyStore** was developed as a dedicated online stationery shopping application that combines modern web technologies with the core functionalities of an e-commerce system, offering a focused and organized solution for stationery shopping.

Chapter 4: TECHNICAL DESCRIPTION

4.1 Hardware Requirements

The hardware requirements for developing and running the StudyStore application are as follows:

Hardware	Specification
Processor	Intel Core i3 or Higher
RAM	4 GB Minimum (8 GB Recommended)
Storage	10 GB Free Disk Space
Display	1366 × 768 Resolution or Higher
Internet Connection	Required

4.2 Software Requirements

The following software and technologies were used for the development of the StudyStore application.

Software / Technology	Version	Purpose
Windows 10/11	64-bit	Operating System
Visual Studio Code	Latest	Code Editor
React.js + Vite	19.x + 7.x	Frontend Development
HTML5, CSS3, JavaScript (ES6+)	Latest	User Interface Development
Node.js + Express.js	22.x + 5.x	Backend Development
MongoDB Atlas + Mongoose	Cloud + 8.x	Database Management
JWT + bcryptjs	9.x + 3.x	Authentication & Password Encryption
Nodemailer	7.x	Email Service
Postman	Latest	API Testing
Git + GitHub	Latest	Version Control

4.3 Development Environment

The StudyStore application is developed using the **MERN Stack**, where each technology performs a specific role in the development process. The frontend, backend, database, and supporting tools work together to provide a complete web application environment.

Development Environment

- **Frontend:** React.js with Vite
- **Backend:** Node.js and Express.js
- **Database:** MongoDB Atlas with Mongoose
- **Authentication:** JWT (JSON Web Token)
- **Password Encryption:** bcryptjs
- **Email Service:** Nodemailer
- **Code Editor:** Visual Studio Code
- **API Testing:** Postman
- **Version Control:** Git and GitHub

The combination of these technologies provides a structured environment for developing, testing, and maintaining the StudyStore.

4.4 Summary

StudyStore is developed using the MERN Stack and supporting development tools to build a complete online stationery shopping system. The selected hardware and software requirements provide a suitable environment for application development, testing, and deployment. This technology stack also allows the application to be maintained and enhanced with additional features in the future.

Chapter 5: SYSTEM DESIGN AND DEVELOPMENT

➤ Introduction

System design represents the overall structure and workflow of the StudyStore application. It includes Software Engineering diagrams, database design, menu design, screen design, and code snippets to explain the implementation of the system.

➤ Flowchart

5.1 Customer Flowchart

Purpose

The Customer Flowchart illustrates the sequence of activities performed by a customer while using the StudyStore application

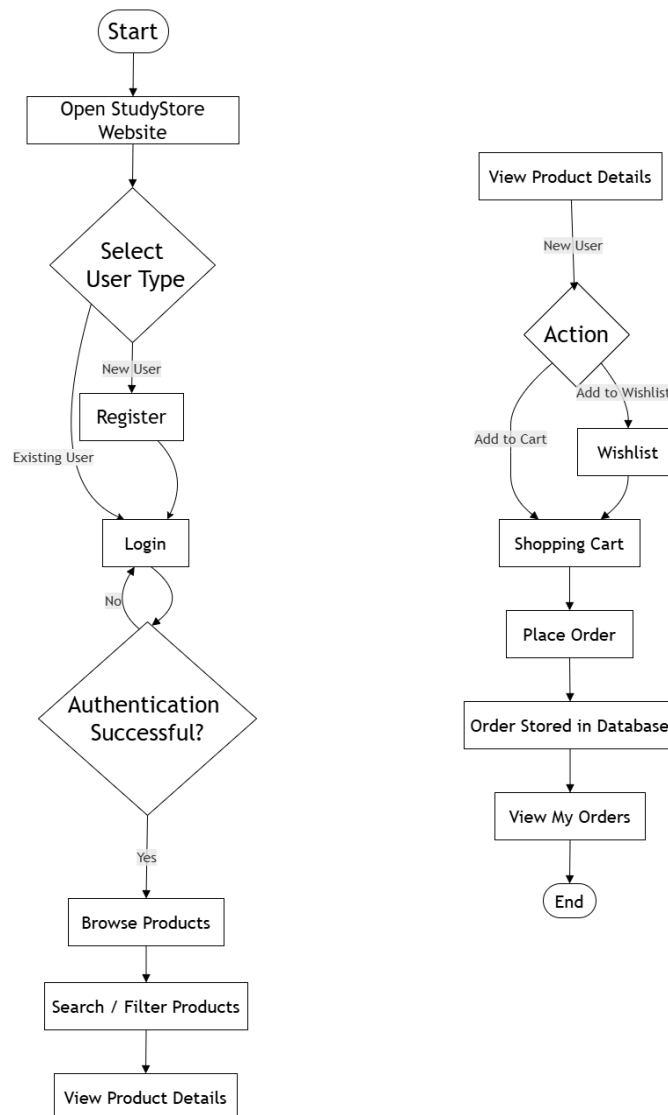


Figure 5.1: Customer Flowchart of StudyStore

Description

The flowchart begins with user authentication, followed by product browsing, searching, adding products to the wishlist or shopping cart, and placing an order. The administrator manages products and customer orders through the admin panel. The diagram provides a clear overview of the complete workflow of the system.

➤ Flowchart

Purpose

The Administrator Flowchart illustrates the sequence of activities performed by the administrator to manage the StudyStore system.

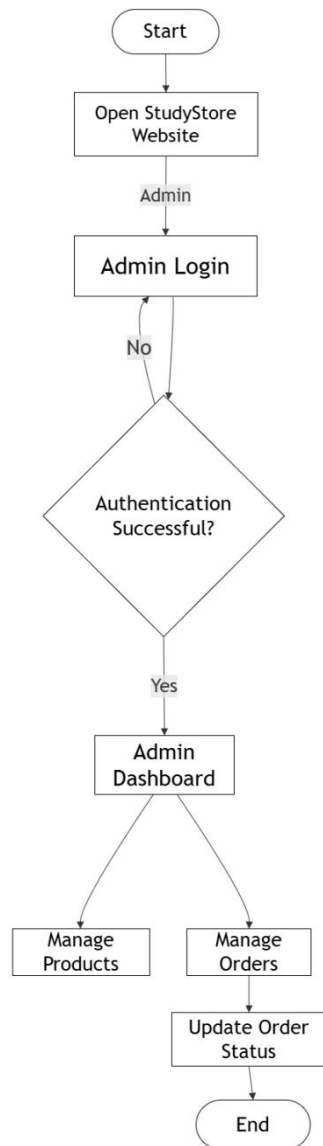


Figure 5.2: Administrator Flowchart of StudyStore

Description

The flowchart begins when the administrator opens the StudyStore website and logs into the system. After successful authentication, the administrator accesses the dashboard to manage products and customer orders. The administrator can update the order status and maintain product information to ensure the smooth operation of the application.

➤ Data Flow Diagram (Context Diagram)

Purpose

The Context Diagram presents a high-level view of the StudyStore system and its interaction with external entities.



Figure 5.3: Context Diagram of StudyStore

Description

The diagram shows the Customer and Administrator interacting with the StudyStore system. Customers perform activities such as registration, login, browsing products, and placing orders, while administrators manage products and orders. The system exchanges information with the MongoDB database to process these operations.

➤ Data Flow Diagram (Level 1)

Purpose

The Level 1 DFD illustrates the major functional processes of the StudyStore application and the movement of data between these processes and the database.

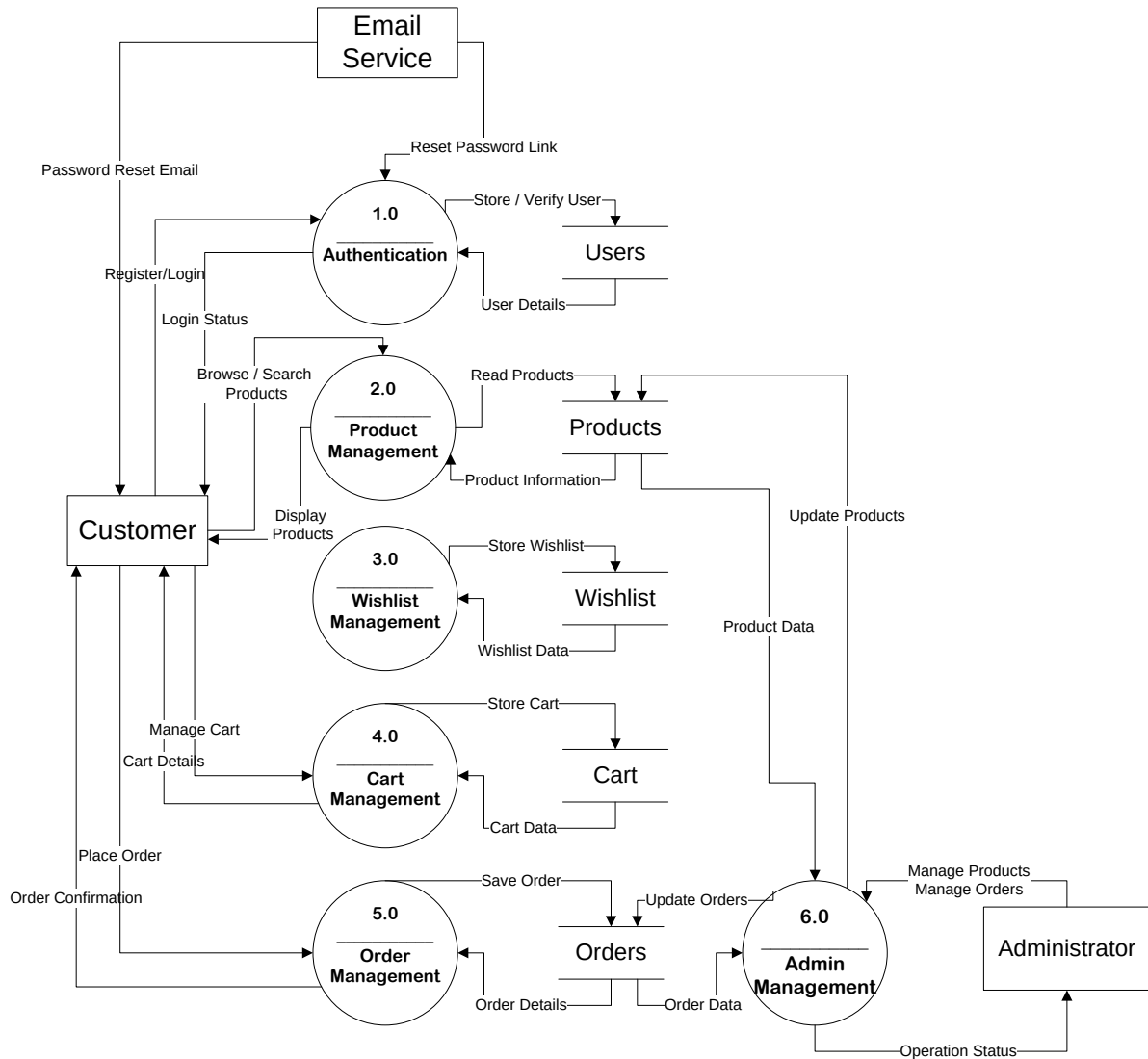


Figure 5.4: Level 1 Data Flow Diagram of StudyStore

Description

The diagram divides the StudyStore system into modules such as Authentication, Product Management, Wishlist Management, Cart Management, and Order Management. It also shows how each module interacts with the MongoDB database to store and retrieve information required for system operations.

➤ Class Diagram

Purpose

The Class Diagram represents the static structure of the StudyStore system by showing its main classes, attributes, methods, and relationships.

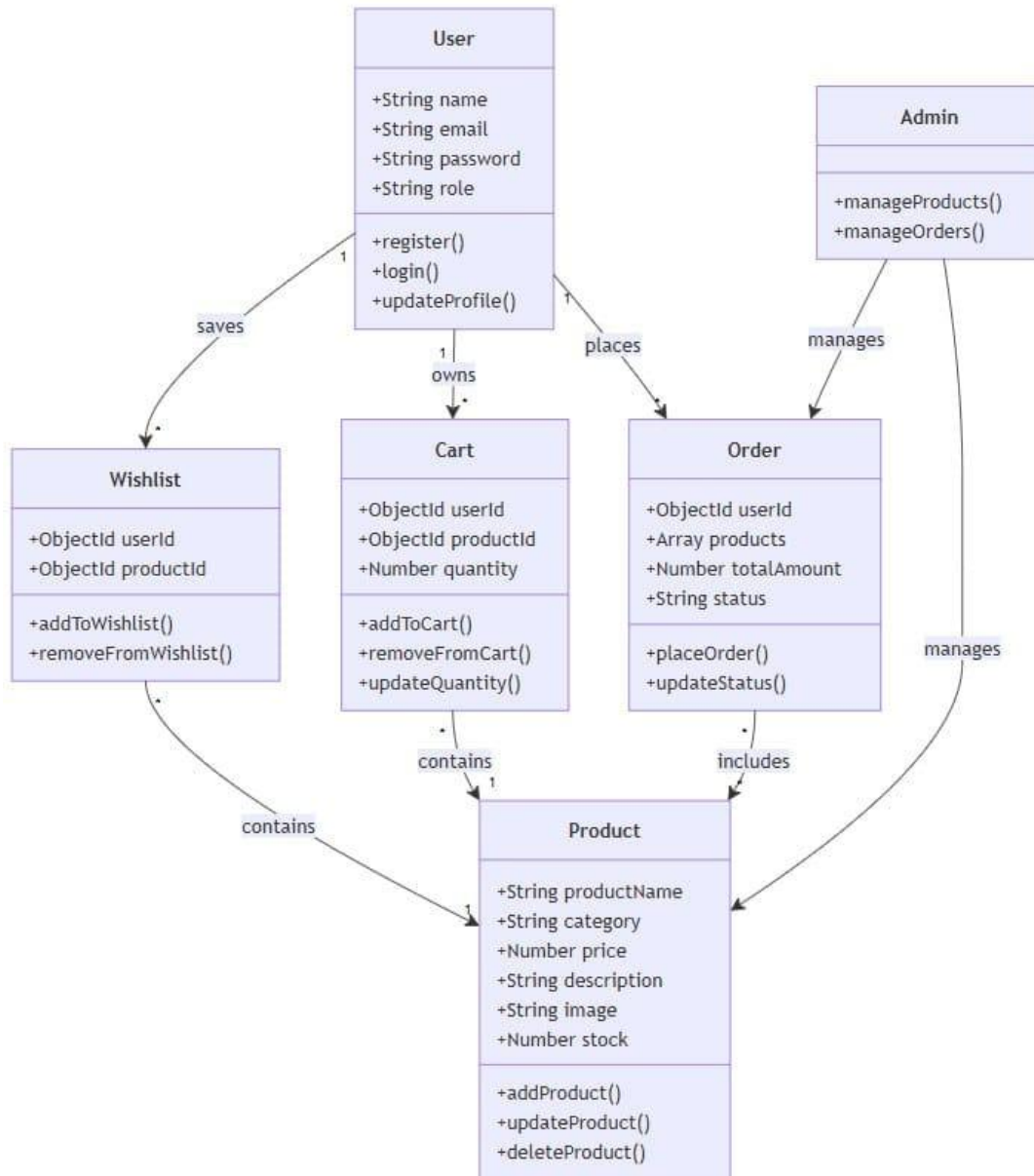


Figure 5.5: Class Diagram of StudyStore

Description

The diagram includes classes such as **User**, **Product**, **Wishlist**, **Cart**, **Order**, and **Admin**. These classes define the core structure of the application and demonstrate how different objects are related. The Class Diagram serves as the foundation for the object-oriented design of the system.

➤ Use Case Diagram

Purpose

The Use Case Diagram illustrates the functional requirements of the StudyStore system and shows how different users interact with the application.

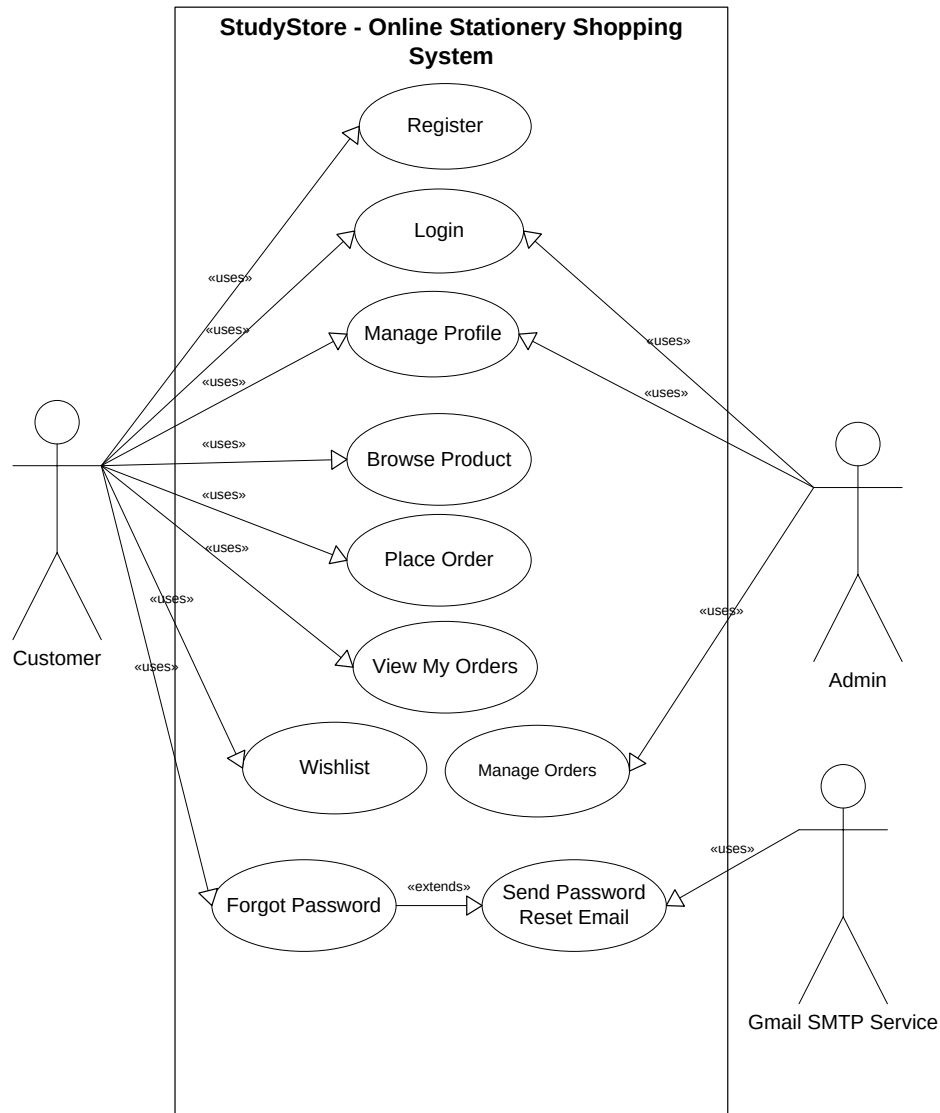


Figure 5.6: Use Case Diagram of StudyStore

Description

The Use Case Diagram illustrates the interaction between the **primary actors**, namely the **Customer** and **Administrator**, and the **secondary actor**, **Gmail SMTP Service**, with the StudyStore system. The Customer can register, log in, manage their profile, browse products, place orders, view order history, and use the forgot password feature. The **Forgot Password** use case extends the **Reset Password** process, where the Gmail SMTP Service sends a password reset link to the registered email address. The Administrator manages products and customer orders to ensure the smooth operation of the system.

➤ Sequence Diagram

Purpose

The Sequence Diagram illustrates the order of interactions between users and the system while performing different operations.

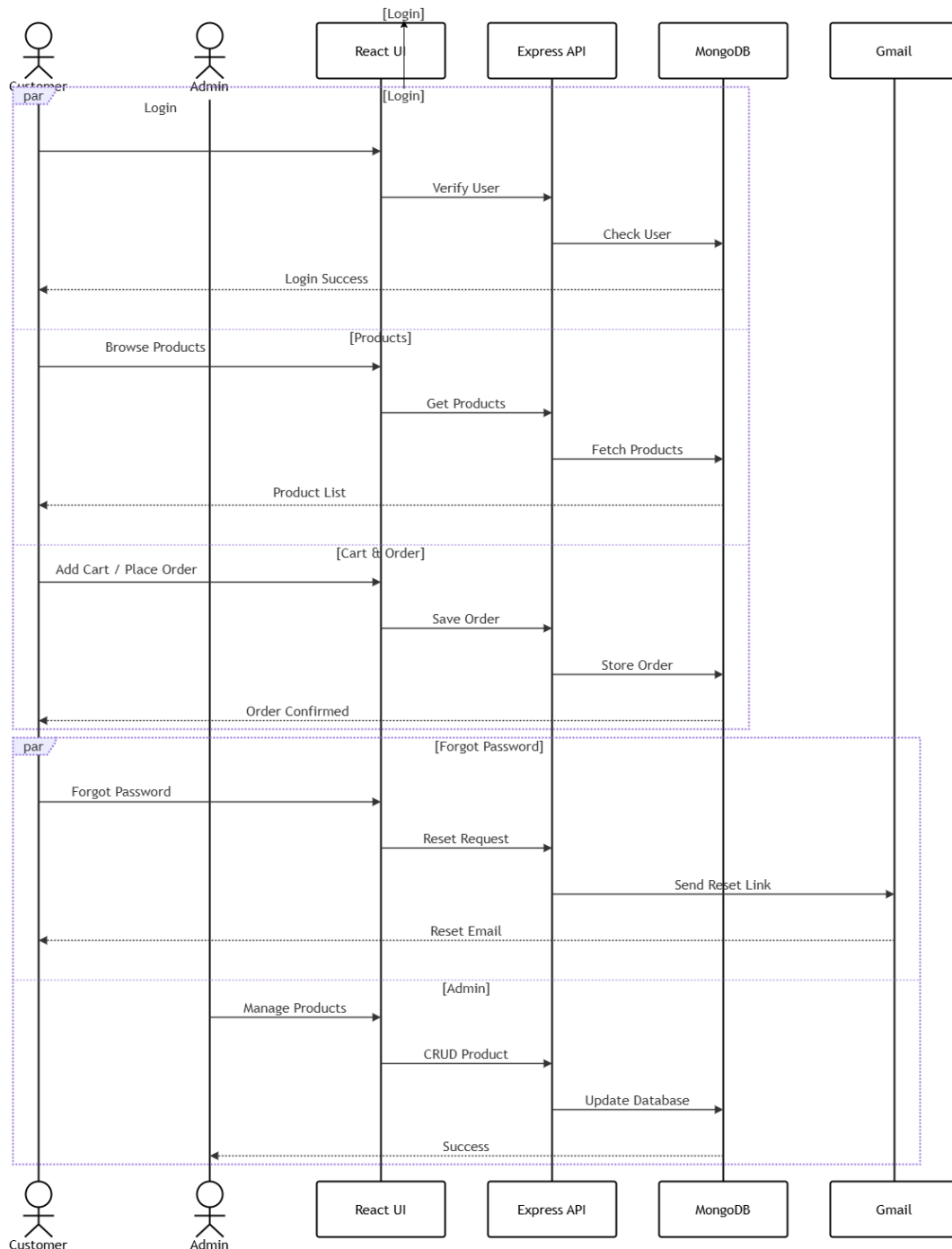


Figure 5.7: Sequence Diagram of StudyStore

Description

The diagram shows the communication between the Customer, Administrator, StudyStore system, MongoDB database, and Email Service. It includes activities such as user login, product browsing, cart management, order placement, password recovery, and product management. The diagram helps in understanding how requests are processed and how different components of the system communicate with each other.

➤ Activity Diagram

5.8 Customer Activity Diagram

Purpose

The Customer Activity Diagram represents the workflow performed by a customer in the StudyStore application. It illustrates the complete shopping process, from user registration and login to browsing products, managing the wishlist and shopping cart, placing an order, and viewing order history.

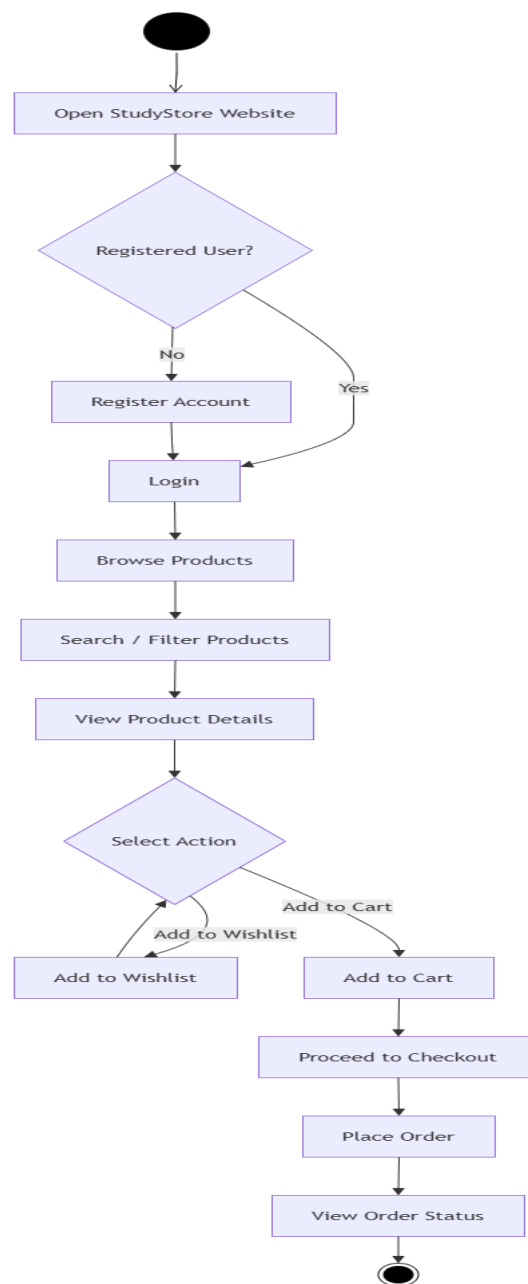


Figure 5.8: Customer Activity Diagram of StudyStore

Description

The Customer Activity Diagram begins with the user opening the StudyStore application and registering or logging in. After successful authentication, the customer browses and searches for stationery products, views product details, and chooses to add products to the wishlist or shopping cart. The customer then places an order, which is stored in the database, and can later view the order details through the **My Orders** section. The activity diagram represents the complete customer shopping workflow within the StudyStore application.

➤ Administrator Activity Diagram

Purpose

The Administrator Activity Diagram represents the workflow performed by the administrator in the StudyStore application. It illustrates the process of administrator login, product management, order management, and logout.

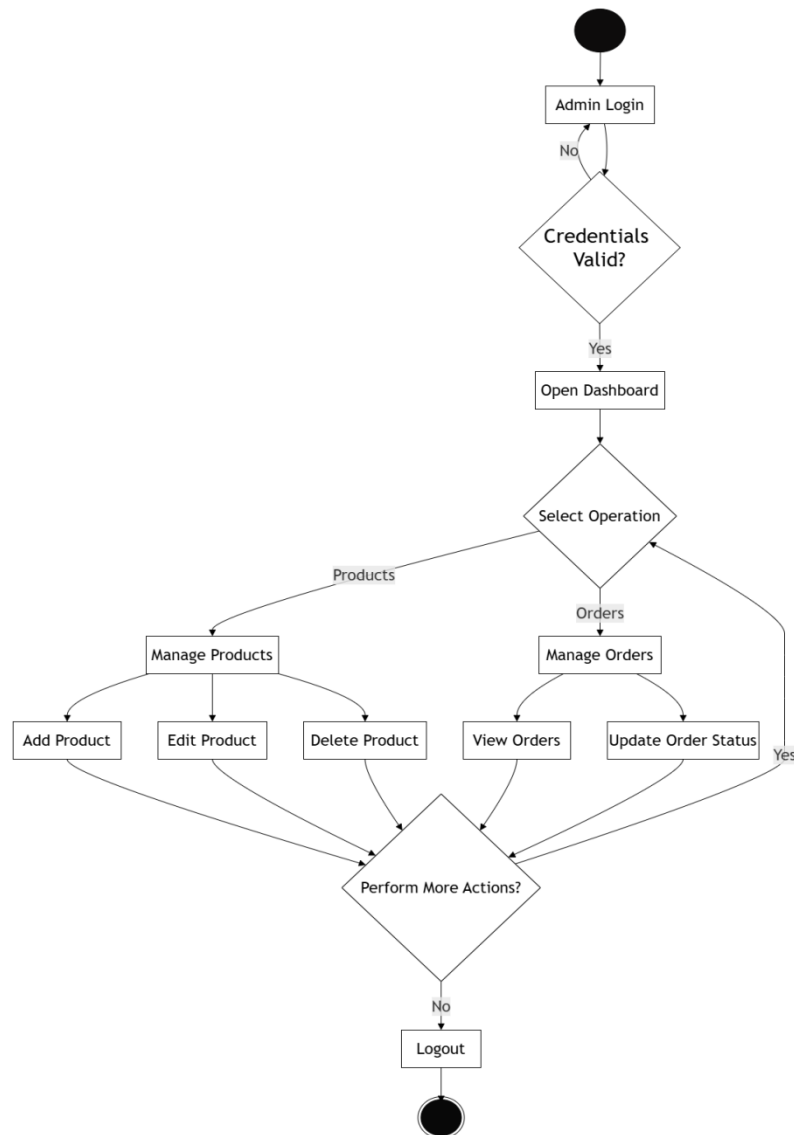


Figure 5.9: Administrator Activity Diagram of StudyStore

Description

The Administrator Activity Diagram begins with the administrator logging into the system. After successful authentication, the administrator accesses the dashboard and selects either product management or order management. The administrator can add, edit, or delete products, view customer orders, and update order status. After completing the required operations, the administrator can either continue performing additional tasks or log out to end the session.

Database Structure (MongoDB Collections)

➤ Introduction

The StudyStore – Online Stationery Shopping System uses **MongoDB Atlas** as its database management system. Since MongoDB is a NoSQL database, data is stored in **collections** instead of traditional relational tables. Each collection is designed to store a specific type of information, such as user accounts, products, shopping carts, wishlists, and orders. The collections are connected using **ObjectId references**, which help maintain relationships between different entities while ensuring efficient data retrieval and management.

5.1 Users Collection

Purpose

The **Users** collection stores information about all registered users and administrators. It is responsible for maintaining user credentials, personal details, profile information, and role-based access required for authentication and authorization.

Field Name	Data Type	Description
_id	ObjectId	Unique identifier of the user
name	String	Full name of the user
email	String	Unique email address
password	String	Encrypted password using bcryptjs
phone	String	Contact number
Address	String	Delivery address
Role	String	User role (User/Admin)
createdAt	Date	Account creation date
updatedAt	Date	Last profile update date

5.2 Wishlist Collection

Purpose

The **Wishlist** collection stores products that users save for future purchase. It allows users to quickly access their preferred stationery items without searching again.

Field Name	Data Type	Description
_id	ObjectId	Unique wishlist identifier
userId	ObjectId	Reference to the user
productId	ObjectId	Reference to the product
createdAt	Date	Date when the product was added

5.3 Products Collection

Purpose

The **Products** collection stores all stationery products available in the StudyStore application. It contains complete product information required for displaying products and managing inventory.

Field Name	Data Type	Description
_id	ObjectId	Unique product identifier
productName	String	Name of the product
description	String	Product description
category	String	Product category
price	Number	Product price
image	String	Product image path or URL
stock	Number	Available product quantity
createdAt	Date	Product creation date
updatedAt	Date	Last product update date

5.4 Cart Collection

Purpose

The **Cart** collection stores products selected by users before placing an order. It also maintains the selected quantity and total amount for each product.

Field Name	Data Type	Description
_id	ObjectId	Unique cart identifier
userId	ObjectId	Reference to the user
productId	ObjectId	Reference to the product
quantity	Number	Quantity selected by the user
totalPrice	Number	Total price of the selected quantity
createdAt	Date	Date when the product was added to the cart

5.5 Orders Collection

Purpose

The **Orders** collection stores complete information about customer orders, including ordered products, shipping details, payment method, and order status.

Field Name	Data Type	Description
_id	ObjectId	Unique order identifier
userId	ObjectId	Reference to the user
products	Array	List of ordered products
totalAmount	Number	Total amount of the order
shippingAddress	String	Delivery address
paymentMethod	String	Payment method (Cash on Delivery)
orderStatus	String	Current order status
orderDate	Date	Date of order placement

➤ Collection Relationships

The following relationships exist between the MongoDB collections used in the StudyStore system.

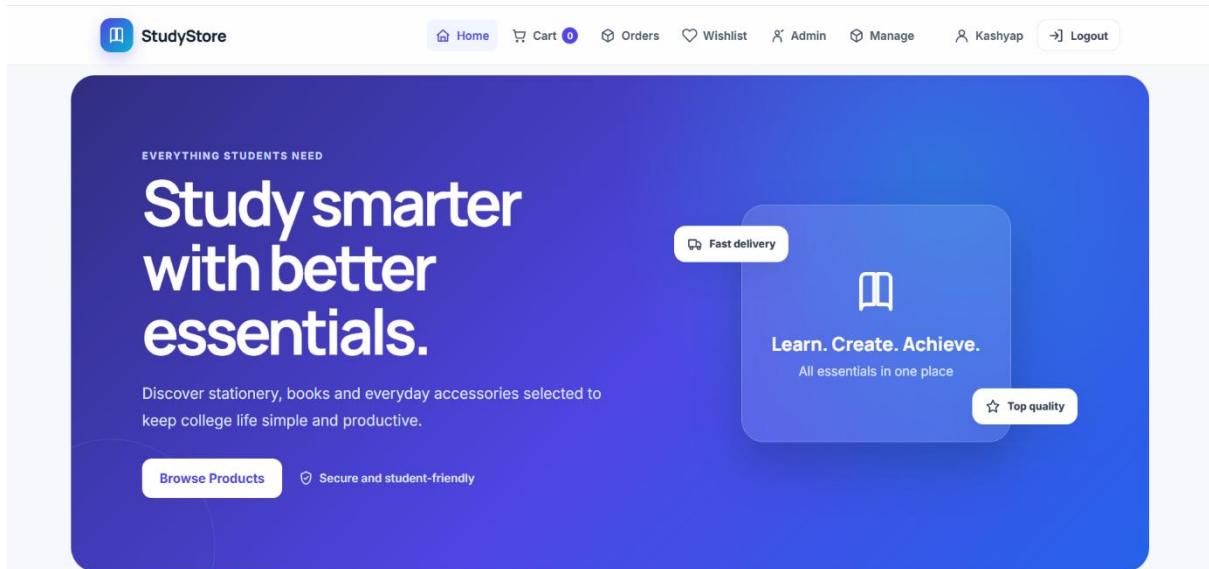
Parent Collection	Child Collection	Relationship
Users	Wishlist	One user can save multiple wishlist items
Users	Cart	One user can have multiple cart items
Users	Orders	One user can place multiple orders
Products	Wishlist	One product can appear in multiple wishlists
Products	Cart	One product can appear in multiple shopping carts
Products	Orders	One product can be included in multiple orders

➤ Database Summary

The StudyStore database is developed using **MongoDB Atlas**, where data is stored in separate collections such as **Users, Products, Wishlist, Cart, and Orders**. Each collection manages a specific type of information, making the system well-organized and easy to maintain. The database supports secure user management, product handling, shopping operations, and order processing while ensuring efficient data storage and retrieval.

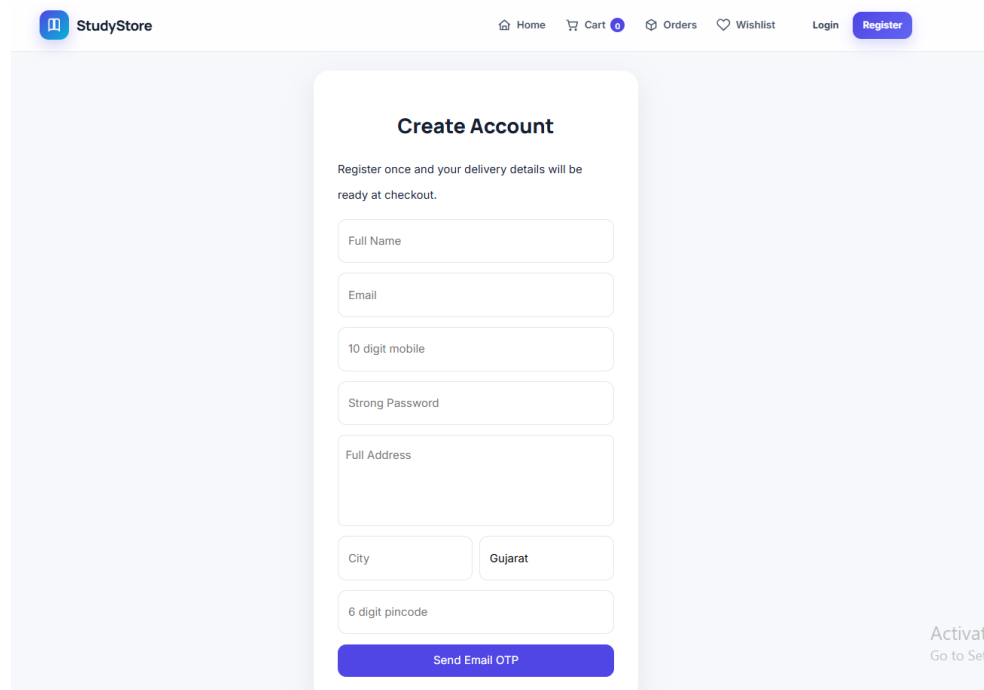
➤ Screen Design

The Screen Design section presents the main user interfaces of the **StudyStore** application and shows how users interact with its key features.



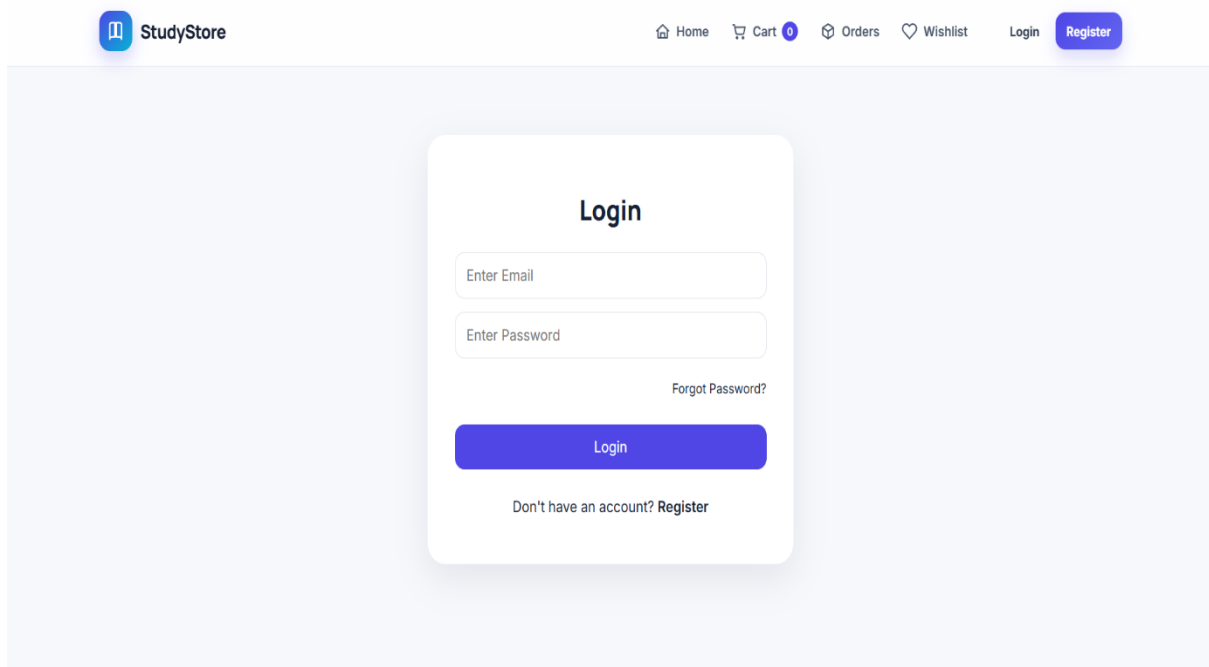
5.10 Home Screen

The Home Screen provides an overview of StudyStore and gives users quick access to products and major shopping features.



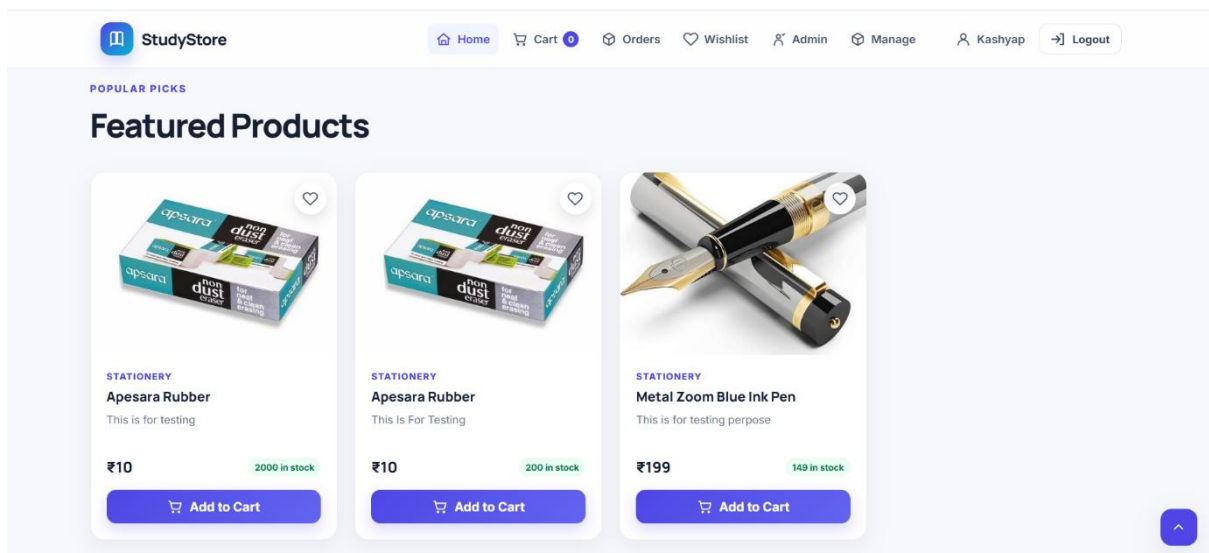
5.11 Registration Screen

The Registration Screen allows new customers to create an account by entering their required personal and login details.



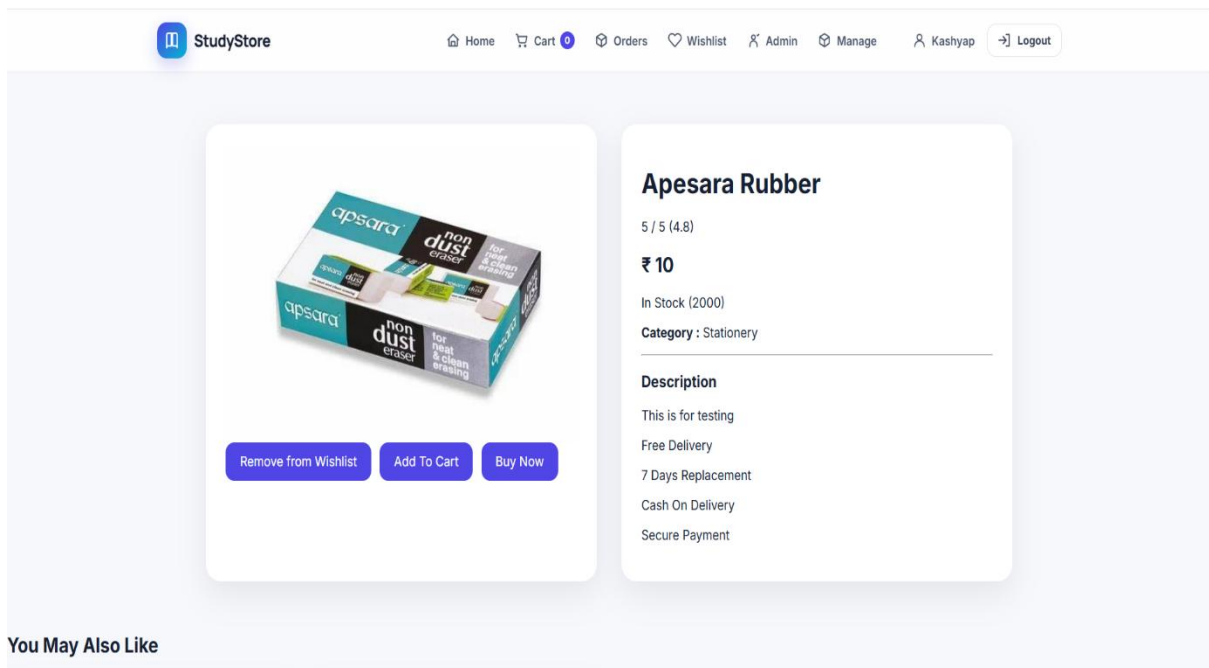
5.12 Login Screen

The Login Screen allows registered users and administrators to access the application using their account credentials.



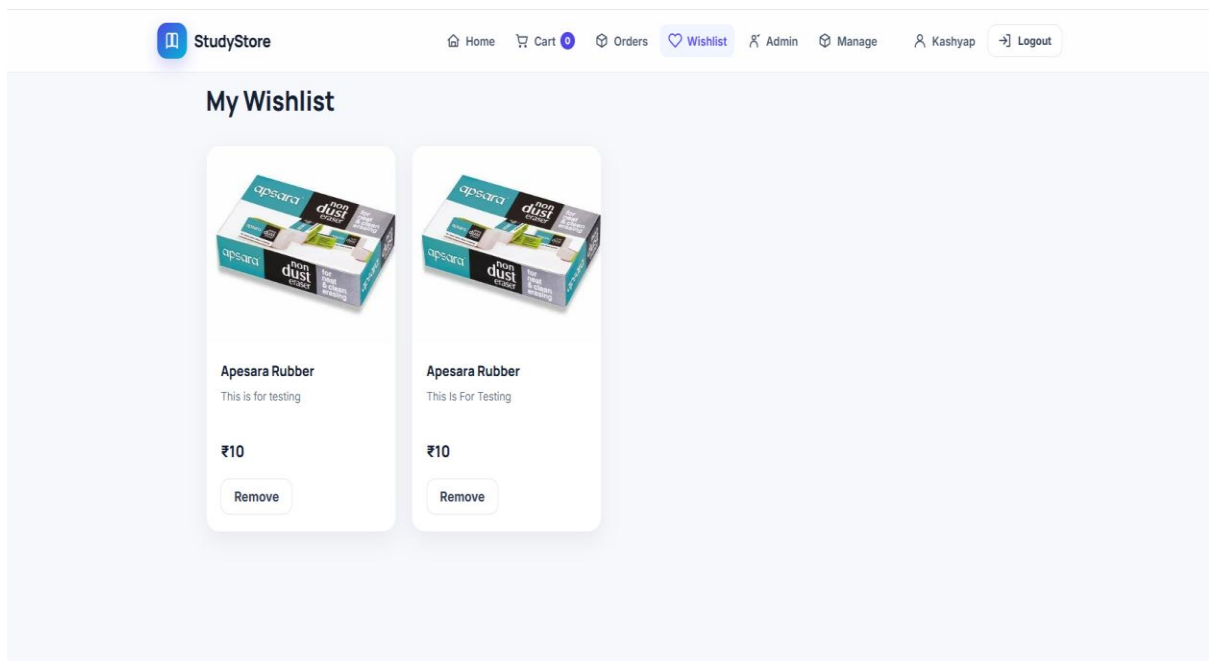
5.13 Product Listing Screen

The Product Listing Screen displays available stationery products and allows customers to browse and search for required items.



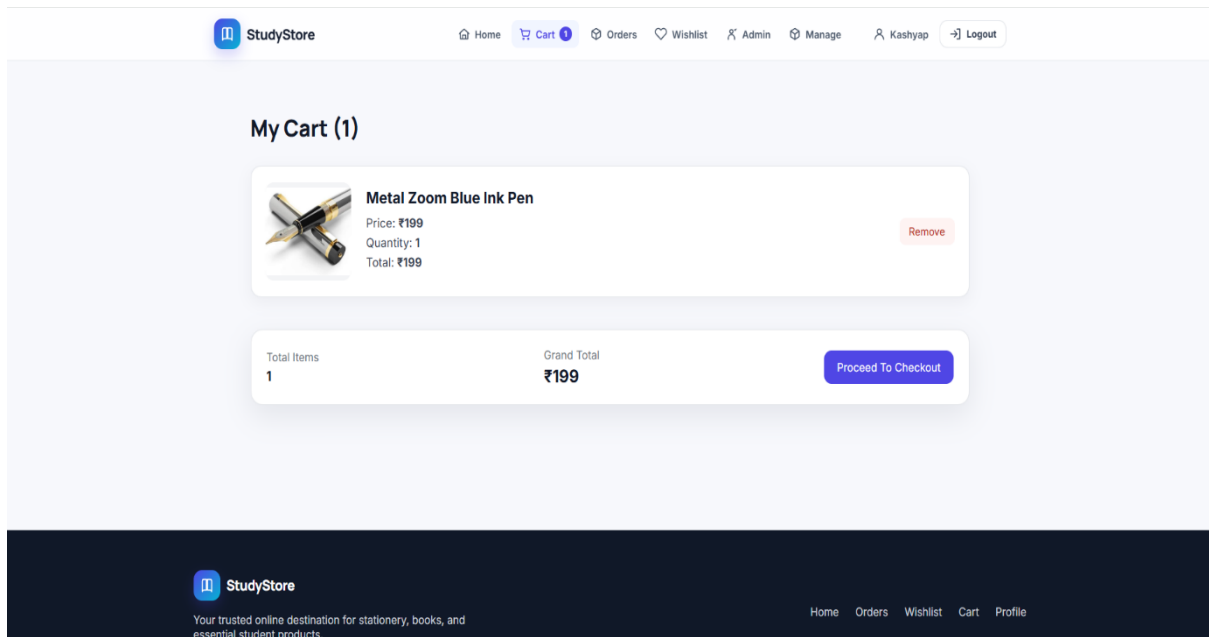
5.14 Product Details Screen

The Product Details Screen presents detailed information about a selected stationery product, including its price, description, image, and availability.



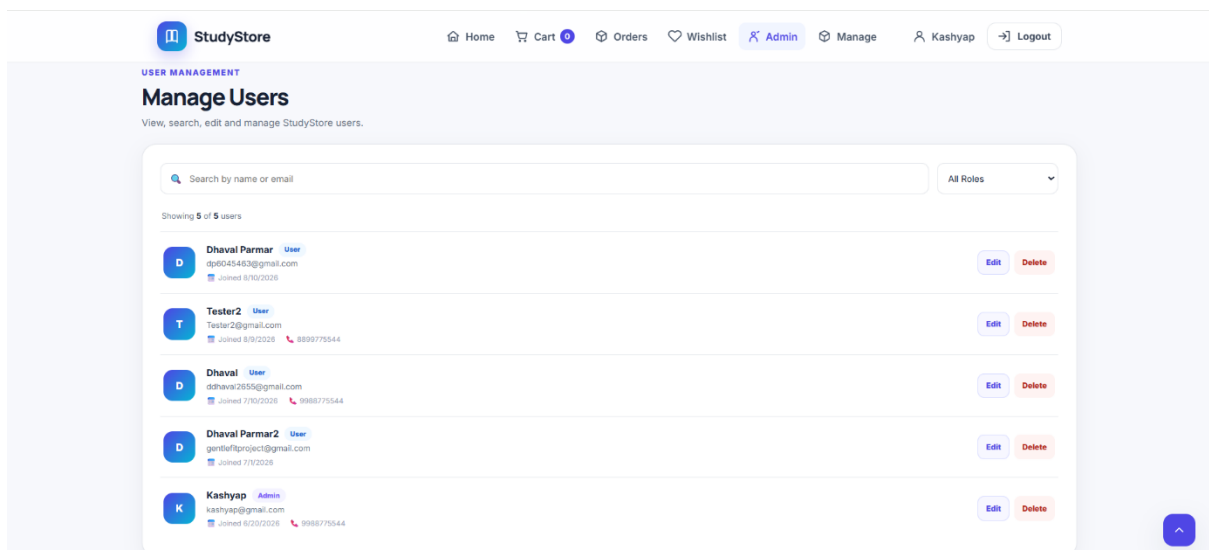
5.15 Wishlist Screen

The Wishlist Screen displays products saved by the customer for future purchase and provides options to manage saved items.



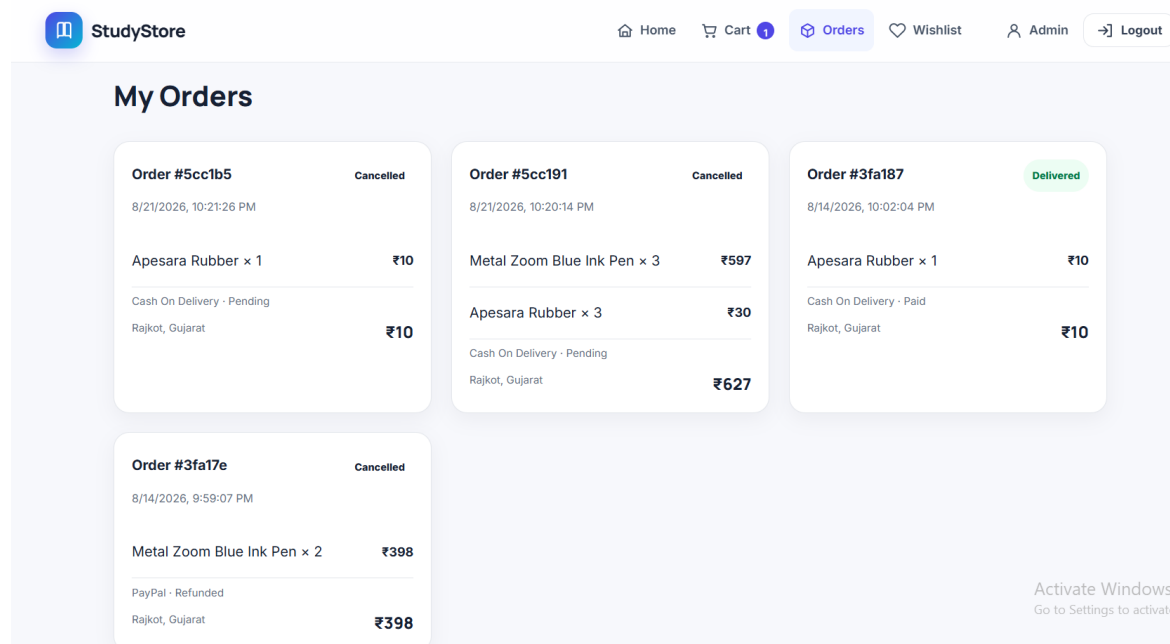
5.16 Shopping Cart Screen

The Shopping Cart Screen displays selected products, quantities, and the total amount before the customer places an order.



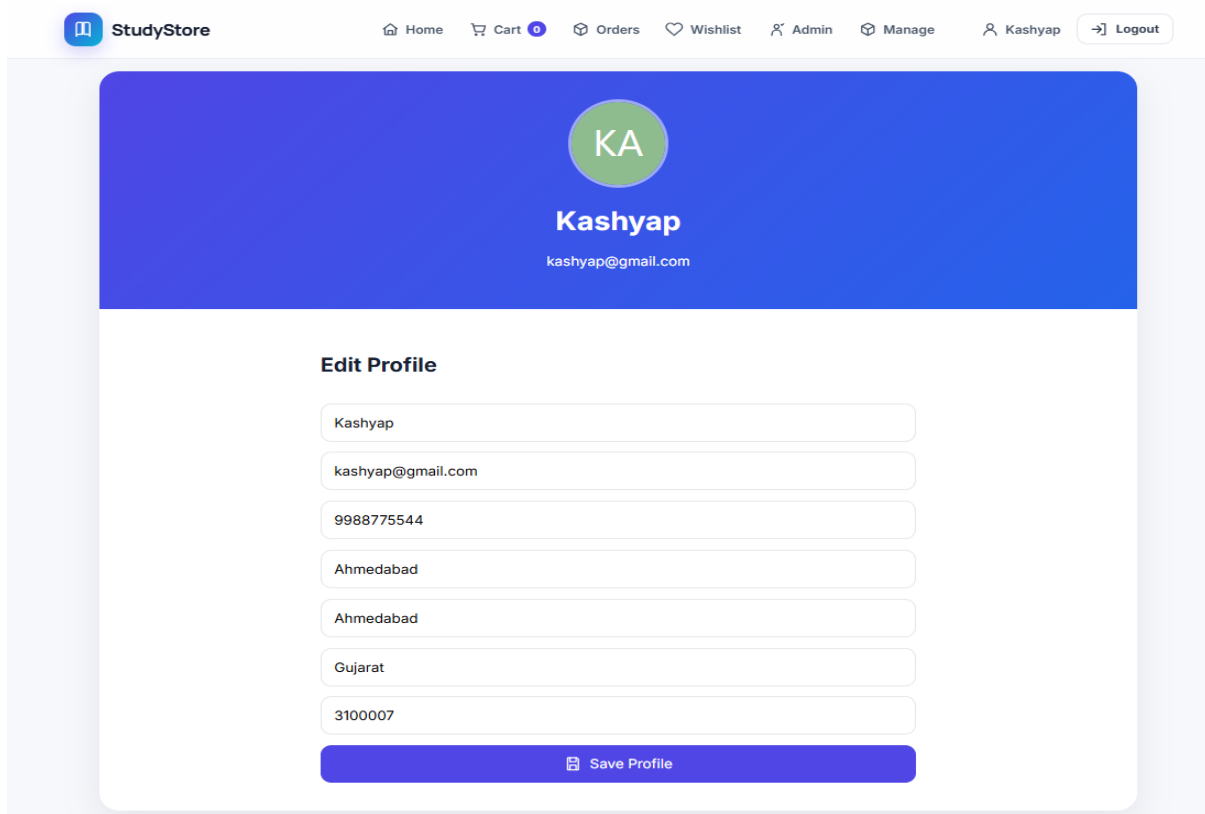
5.17 User Management Screen

The User Management Screen allows administrators to view and manage registered user information and account details.



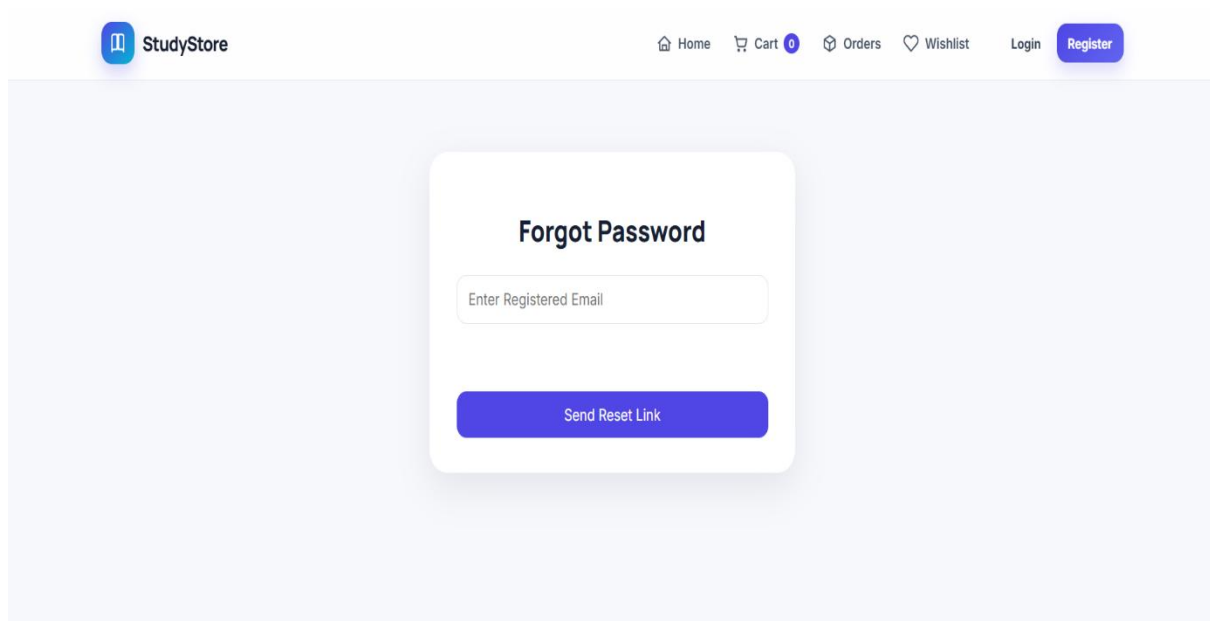
5.18 My Orders Screen

The My Orders Screen allows customers to view their previous orders and check the current status of each order.



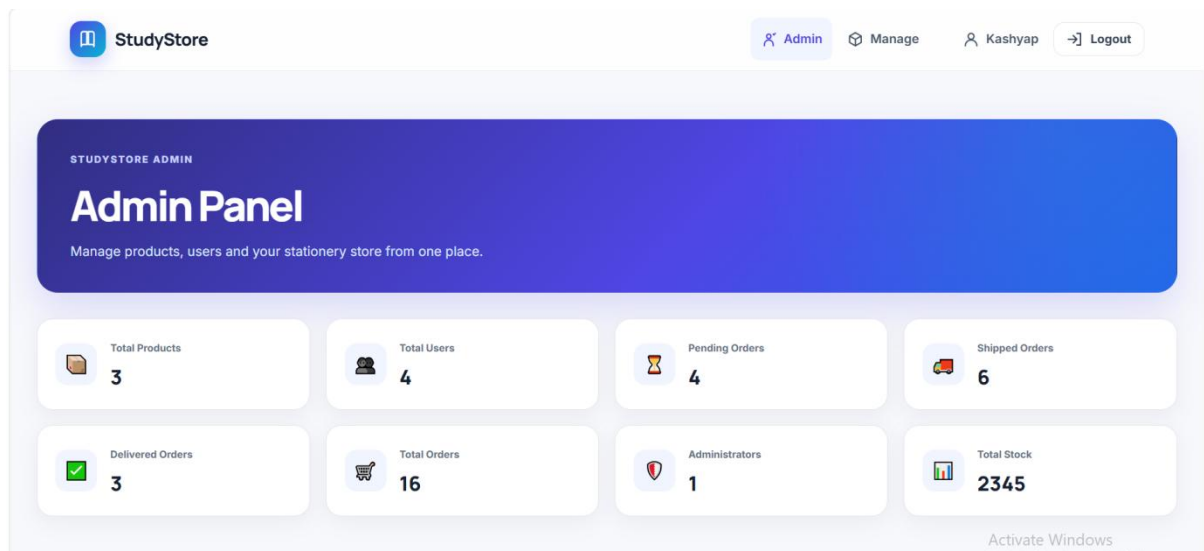
5.19 Profile Screen

The Profile Screen allows customers to view and update their personal account information.



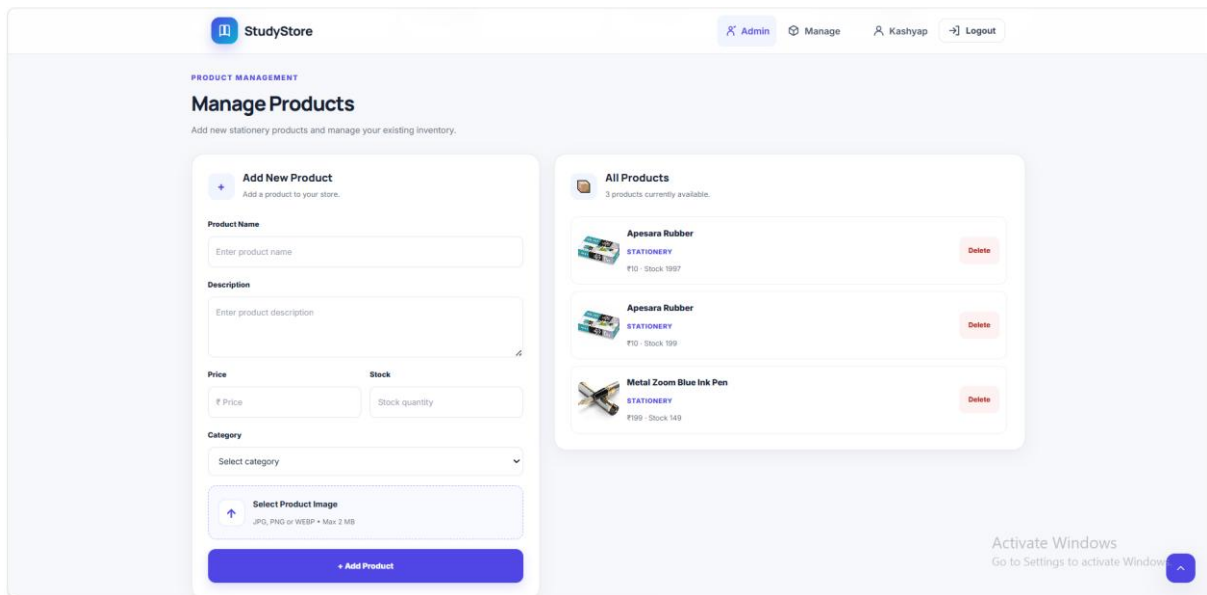
5.20 Forgot Password Screen

The Forgot Password Screen allows users to request a password recovery link by providing their registered email address.



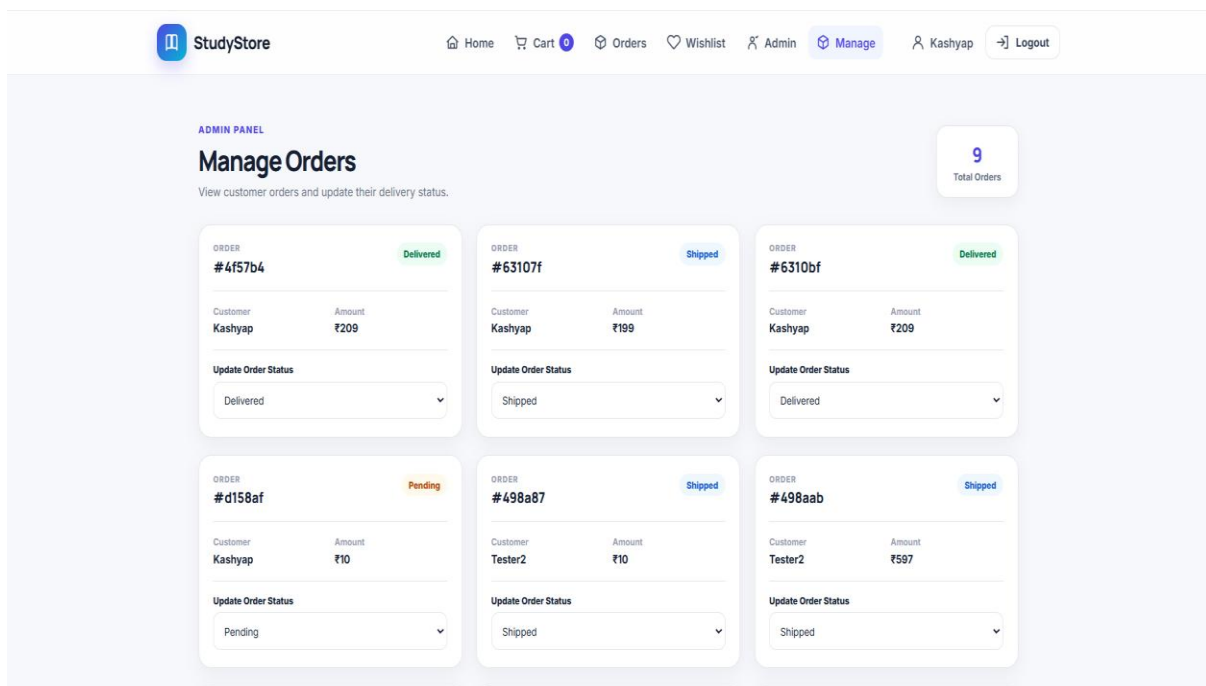
5.21 Admin Dashboard Screen

The Admin Dashboard provides administrators with an overview and access to the main store management functions.



5.22 Product Management Screen

The Product Management Screen allows administrators to add, edit, and delete stationery products.



5.23 Order Management Screen

The Order Management Screen allows administrators to view customer orders and update their order status.

Figure 5.24: Coupon Management

The Coupon screen allows users to apply an available coupon during the purchase process and receive the applicable discount on their order.

Figure 5.25: Checkout Screen

The Checkout screen allows users to review their order details, enter delivery information, select a payment method, and proceed with the purchase.

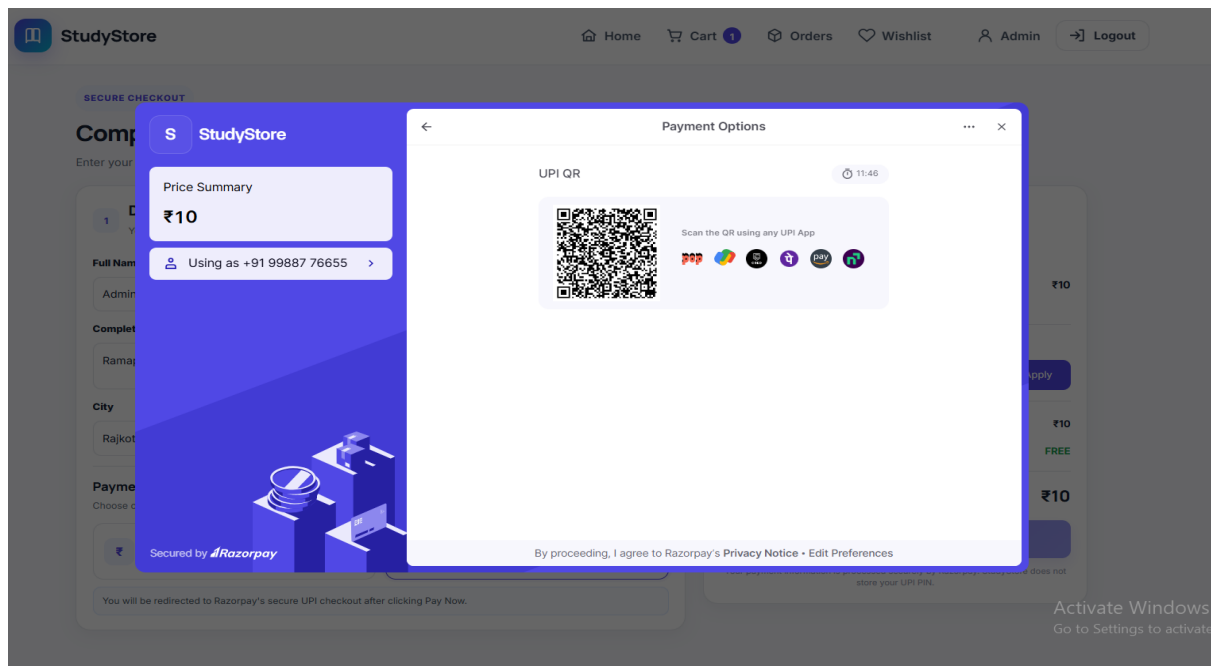


Figure 5.26: Razorpay UPI Payment Screen

The Razorpay UPI Payment screen allows users to complete their order payment conveniently through UPI.

➤ Code of the Modules

This section presents selected code snippets from the major modules of the **StudyStore** application. The code highlights important implementation logic used for database connectivity, user authentication, product handling, shopping operations, and order management.

5.1 Database Connection

Purpose:

This code establishes the connection between the StudyStore backend and the MongoDB Atlas database.

Code:

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);

    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error("Database Connection Error:", error.message);
    process.exit(1);
  }
}; module.exports = connectDB;
```

Description:

The database connection enables the backend to store and retrieve application data from MongoDB Atlas.

5.2 User Registration

Purpose:

This module handles the creation of new customer accounts.

Code:

```
const registerUser = async (req, res) => {

  try {

    const { name, email, password } = req.body;

    // Password validation
```

```
const passwordError = validatePassword(password);

if (passwordError) {
  return res.status(400).json({
    message: passwordError
  });
}

const userExists = await User.findOne({ email });

if (userExists) {
  return res.status(400).json({
    message: "User already exists"
  });
}

const hashedPassword = await bcrypt.hash(
  password,
  10
);

const user = await User.create({
  name,
  email,
  password: hashedPassword
});

res.status(201).json({
  message: "User registered successfully",
  user
});

} catch (error) {
  res.status(500).json({
    message: "Internal Server Error"
  });
}
};
```

Description:

The registration process validates user information, encrypts the password using bcryptjs, and stores the account details in the database.

5.3 User Login and JWT Authentication

Purpose:

This module verifies user credentials and manages authenticated access.

Code:

```
const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) {
      return res.status(400).json({
        message: "Invalid Email or Password"
      });
    }
    const isMatch = await bcrypt.compare(
      password,
      user.password
    );
    if (!isMatch) {
      return res.status(400).json({
        message: "Invalid Email or Password"
      });
    }
    const token = jwt.sign(
      {
        id: user._id,
        role: user.role
      },
      process.env.JWT_SECRET,
      {
        expiresIn: "7d"
      }
    );
    res.status(200).json({
      message: "Login Successful",
      token,
      userId: user._id,
      name: user.name,
      role: user.role
    });
  } catch (error) {
    console.log(error);
    res.status(500).json({
      message: error.message
    });
  }
};
```

Description:

The login process checks the submitted credentials and generates a JWT token after successful authentication. The token is used to access protected features.

5.4 Product Management

Purpose:

This module handles the creation and management of stationery products.

Code:

```
const createProduct = async (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({
        message: "Product image is required",
      });
    }
    const productData = {
      ...req.body,
      image: `uploads/products/${req.file.filename}`,
    };
    if (productData.price) {
      productData.price = Number(productData.price);
    }
    if (productData.stock) {
      productData.stock = Number(productData.stock);
    }
    const product = await Product.create(productData);
    res.status(201).json({
      message: "Product Added Successfully",
      product,
    });
  } catch (error) {
    console.error("Create product error:", error);

    if (req.file) {
      const uploadedFilePath = path.join(
        __dirname,
        "..",
        "uploads",
        "products",
        req.file.filename
      );
      if (fs.existsSync(uploadedFilePath)) {
        fs.unlinkSync(uploadedFilePath);
      }
    }
    res.status(500).json({
      message: error.message,
    });
  }
};
```

Description:

The module allows product information to be added, retrieved, updated, or removed according to the administrator's operations.

5 .5 Wishlist Management

Purpose:

This module manages products saved by customers for future purchase.

Code:

```
const Wishlist = require("../models/Wishlist");

const toggleWishlist = async (req, res) => {
  try {
    const { user, product } = req.body;
    if (!user || !product) return res.status(400).json({ message: "User and product are required" });

    const existing = await Wishlist.findOne({ user, product });
    if (existing) {
      await existing.deleteOne();
      return res.json({ message: "Removed from Wishlist", active: false, itemId: existing._id });
    }

    const item = await Wishlist.create({ user, product });
    const populated = await item.populate("product");
    return res.status(201).json({ message: "Added to Wishlist", active: true, item: populated });
  } catch (error) {
    if (error.code === 11000) return res.status(409).json({ message: "Already in Wishlist", active: true });
    res.status(500).json({ message: error.message });
  }
};

const getWishlist = async (req, res) => {
  try {
    const filter = req.query.user ? { user: req.query.user } : {};
    const items = await Wishlist.find(filter).populate("product").sort({ createdAt: -1 });
    res.json(items);
  } catch (error) { res.status(500).json({ message: error.message }); }
};

const removeWishlist = async (req, res) => {
  try {
    const item = await Wishlist.findByIdAndDelete(req.params.id);
    if (!item) return res.status(404).json({ message: "Wishlist item not found" });
    res.json({ message: "Removed", active: false });
  } catch (error) { res.status(500).json({ message: error.message }); }
};

module.exports = { toggleWishlist, getWishlist, removeWishlist };
```

Description:

The module adds selected products to the customer's wishlist and allows saved items to be retrieved or removed.

5.6 Shopping Cart Management

Purpose:

This module manages products selected by customers before ordering.

Code:

```
exports.addToCart = async (req, res) => {
  try {
    const user = req.user.id,
      product = req.body.product,
      add = Math.max(1, Number(req.body.quantity) || 1);
    const p = await Product.findById(product);
    if (!p) return res.status(404).json({ message: "Product not found" });
    let item = await Cart.findOne({ user, product });
    const qty = (item?.quantity || 0) + add;
    if (qty > p.stock)
      return res
        .status(400)
        .json({ message: `Only ${p.stock} items available` });
    if (item) {
      item.quantity = qty;
      await item.save();
    } else item = await Cart.create({ user, product, quantity: add });
    await item.populate(
      "product",
      "name description price stock category image",
    );
    res.status(201).json({ message: "Cart updated", cartItem: item });
  } catch (e) {
    res.status(500).json({ message: e.message });
  }
};

exports.getCartItems = async (req, res) => {
  try {
    res.json(
      await populate(Cart.find({ user: req.user.id })).sort({ createdAt: -1 }),
    );
  } catch (e) {
    res.status(500).json({ message: e.message });
  }
};

exports.updateQuantity = async (req, res) => {
  try {
    const item = await Cart.findOne({
      _id: req.params.id,
      user: req.user.id,
    }).populate("product");
    if (!item) return res.status(404).json({ message: "Cart item not found" });
    const q = Number(req.body.quantity);
    if (q < 1)
      return res.status(400).json({ message: "Quantity must be at least 1" });
    if (q > item.product.stock)
      return res
        .status(400)
        .json({ message: `Only ${item.product.stock} items available` });
  }
};
```

```

    item.quantity = q;
    await item.save();
    res.json({ message: "Quantity updated", cartItem: item });
  } catch (e) {
    res.status(500).json({ message: e.message });
  }
});
exports.removeCartItem = async (req, res) => {
  const x = await Cart.findOneAndDelete({
    _id: req.params.id,
    user: req.user.id,
  });
  if (!x) return res.status(404).json({ message: "Cart item not found" });
  res.json({ message: "Item removed" });
};
exports.clearCart = async (req, res) => {
  await Cart.deleteMany({ user: req.user.id });
  res.json({ message: "Cart cleared" });
};

```

Description:

The cart module handles adding products, updating quantities, removing items, and calculating the required amount.

5.7 Order Management

Purpose:

This module processes customer orders and maintains order information.

Code:

```

exports.createOrder = async (req, res) => {
  try {
    const items = await Cart.find({
      user: req.user.id,
    }).populate("product");

    if (!items.length) {
      return res.status(400).json({
        message: "Cart is empty",
      });
    }

    let subtotal = 0;

    for (const item of items) {
      if (

```

```

        !item.product ||
        item.quantity > item.product.stock
    ) {
        return res.status(400).json({
            message: `Insufficient stock for ${
                item.product?.name || "product"
            }`,
        });
    }
    subtotal +=
        Number(item.product.price) *
        Number(item.quantity);
}
exports.getOrders = async (
    req,
    res
) => {
    try {
        const orders = await Order.find()
            .populate(
                "user",
                "name email"
            )
            .populate(
                "products.product",
                "name price image"
            )
            .sort({
                createdAt: -1,
            });
        res.json(orders);
    } catch (error) {
        res.status(500).json({
            message: error.message,
        });
    }
}
exports.cancelOrder = async (
    req,
    res
) => {
    try {
        const order =
            await Order.findOne({
                _id: req.params.id,
                user: req.user.id,
            }).populate("user");

        if (!order) {
            return res.status(404).json({
                message: "Order not found",
            });
        }

        // Customer can cancel only Pending / Confirmed
        if (

```



```

    ![
      "Pending",
      "Confirmed",
    ].includes(order.status)
  ){
    return res.status(400).json({
      message:
        "This order can no longer be cancelled",
    });
  }
}

```

```

order.status = "Cancelled";

```

```

order.cancelledAt = new Date();

```

```

order.cancellationReason =
  req.body.reason ||
  "Cancelled by customer";

```

```

// Refund PayPal payment
if (
  order.paymentStatus === "Paid"
){
  order.paymentStatus =
    "Refunded";
}
await order.save();

```

```

// Restore stock
for (const item of order.products) {
  await Product.findByIdAndUpdate(
    item.product,
    {
      $inc: {
        stock: item.quantity,
      },
    },
  );
}

```

```

// Email
if (order.user) {
  mail(
    order.user.email,
    "StudyStore - Order Cancelled",
    `
      <h2>Order Cancelled</h2>

      <p>
        Hi ${order.user.name},
      </p>
      <p>
        Your order
        <strong>
          #${order._id

```

```

        .toString()
        .slice(-6)}
    </strong>
    has been cancelled.
  </p>
  );
}
res.json({
  message: "Order cancelled",
  order,
});
} catch (error) {
  console.error(
    "Cancel order error:",
    error
  );

  res.status(500).json({
    message: error.message,
  });
}

```

Description:

The module creates orders from selected cart items, stores order details in MongoDB, and provides order information for customers and administrators.

5 .8 Admin Management

Purpose:

This module provides administrators with functions for managing products and customer orders.

Code:

```

// =====
// GET ALL ORDERS - ADMIN
// =====

exports.getOrders = async (
  req,
  res
) => {
  try {
    const orders = await Order.find()
      .populate(
        "user",
        "name email"
      )
      .populate(
        "products.product",

```

```

        "name price image"
    )
    .sort({
        createdAt: -1,
    });

    res.json(orders);
  } catch (error) {
    res.status(500).json({
      message: error.message,
    });
  }
});

// =====
// ADMIN UPDATE ORDER STATUS
// =====

exports.updateOrderStatus = async (
  req,
  res
) => {
  try {
    const allowedStatuses = [
      "Pending",
      "Confirmed",
      "Shipped",
      "Out for Delivery",
      "Delivered",
      "Cancelled",
    ];

    const newStatus =
      req.body.status;

    if (
      !allowedStatuses.includes(
        newStatus
      )
    ) {
      return res.status(400).json({
        message:
          "Invalid status",
      });
    }

    const order =
      await Order.findById(
        req.params.id
      ).populate("user");

    if (!order) {
      return res.status(404).json({

```

```

        message:
            "Order Not Found",
        });
    }
    // -----
    // PREVENT STOCK RESTORING TWICE
    // -----

    const wasCancelled =
        order.status === "Cancelled";

    const isCancelled =
        newStatus === "Cancelled";

    // -----
    // UPDATE STATUS
    // -----

    order.status = newStatus;
    // COD becomes paid when delivered
    if (
        newStatus === "Delivered" &&
        order.paymentMethod ===
            "Cash On Delivery"
    ) {
        order.paymentStatus =
            "Paid";

        order.paidAt =
            new Date();
    }
    // -----
    // CANCELLED BY ADMIN
    // -----

    if (
        isCancelled &&
        !wasCancelled
    ) {
        order.cancelledAt =
            new Date();

        order.cancellationReason =
            req.body.reason ||
            "Cancelled by admin";

        if (
            order.paymentStatus ===
                "Paid"
        ) {
            order.paymentStatus =

```

```
    "Refunded";  
  }  
  
  // Restore stock  
  for (  
    const item of order.products  
  ) {  
    await Product.findByIdAndUpdate(  
      item.product,  
      {  
        $inc: {  
          stock: item.quantity,  
        },  
      },  
      {  
        new: true,  
      }  
    );  
  }  
  await order.save();  
}
```

Description:

The administrator can perform management operations such as modifying products and updating customer order status through the admin interface.

Chapter 6: SYSTEM TESTING

6.1 Introduction

System testing was conducted to evaluate the overall performance of the **StudyStore** application and to ensure that it operates according to the project requirements. During this phase, all major modules were tested together to verify their functionality and interaction. Features such as user authentication, product management, wishlist, shopping cart, order processing, and administrator operations were examined to confirm that the application delivers the expected results and functions smoothly before deployment.

6.2 Testing Objectives

The main objectives of system testing are:

- To verify that all modules function as expected.
- To identify and correct errors before deployment.
- To ensure smooth interaction between different modules.
- To validate user inputs and system responses.
- To verify that the application satisfies the project requirements.
- To improve the overall reliability and performance of the application.

6.3 Test Cases

The following test cases were executed to verify the functionality of the StudyStore application. Each module was tested using valid inputs to ensure that the system produced the expected results.

Test Case ID	Module	Test Scenario	Expected Result	Status
TC-01	User Registration	Register with valid user details	User account is created successfully	Pass
TC-02	User Login	Login with valid email and password	User is logged in successfully	Pass
TC-03	Forgot Password	Enter registered email address	Password reset link is sent to the email	Pass
TC-04	Product Search	Search a product by keyword	Matching products are displayed	Pass
TC-05	Wishlist	Add a product to the wishlist	Product is added to the wishlist	Pass
TC-06	Shopping Cart	Add a product to the shopping cart	Product is added to the cart successfully	Pass
TC-07	Place Order	Place an order from the shopping cart	Order is placed and stored successfully	Pass

TC-08	My Orders	View previously placed orders	Order history is displayed correctly	Pass
TC-09	Product Management	Admin adds, edits, or deletes a product	Product information is updated successfully	Pass
TC-10	Order Management	Admin updates the order status	Updated order status is displayed correctly	Pass

6.4 Testing Summary

The StudyStore application was evaluated by executing test cases for its major modules, including user authentication, product browsing, wishlist, shopping cart, order management, and administrator operations. The observed results matched the expected outcomes for all test cases, demonstrating that each module functions as intended. The successful completion of testing indicates that the application is reliable, performs its required operations effectively, and is suitable for deployment.

Chapter 7: CONCLUSION

7. Conclusion

The development of **StudyStore – Online Stationery Shopping System** successfully transformed the concept of a stationery e-commerce platform into a functional full-stack web application. The project combined software engineering concepts with modern web technologies to create an application that is organized, user-friendly, and easy to manage.

The major outcomes of the project are summarized below:

- Successfully developed a complete online stationery shopping application using the **MERN Stack**.
- Implemented customer features such as registration, login, product browsing, wishlist, shopping cart, and order placement.
- Designed an administrator dashboard for efficient product and order management.
- Applied **JWT authentication**, **bcryptjs password encryption**, and **Nodemailer** for account management.
- Developed a structured **MongoDB Atlas** database for efficient data storage and retrieval.
- Implemented RESTful APIs to enable smooth communication between the frontend and backend.
- Followed Software Engineering practices, including system design, UML diagrams, database design, and testing.
- Strengthened practical knowledge of React.js, Node.js, Express.js, MongoDB Atlas, Git, and API development through real-world implementation.

Overall, the project successfully achieved its objectives and provided valuable experience in full-stack web development, database management, software design, and project documentation. It serves as a strong foundation for developing more advanced e-commerce applications in the future.

Chapter 8: LEARNING DURING PROJECT WORK

8.1 Learning During Project Work

The development of the **StudyStore – Online Stationery Shopping System** provided valuable practical experience in software development and project management. Throughout the project, several technical and professional skills were enhanced.

The major learning outcomes are:

- Gained practical experience in developing a full-stack web application using the **MERN Stack**.
- Learned to design responsive user interfaces using **React.js** and **Vite**.
- Improved backend development skills using **Node.js** and **Express.js**.
- Acquired hands-on experience with **MongoDB Atlas** for cloud database management.
- Understood the implementation of **RESTful APIs** for communication between frontend and backend.
- Learned to implement **JWT-based authentication**, password encryption using **bcryptjs**, and password recovery using **Nodemailer**.
- Improved knowledge of **Git** and **GitHub** for version control and project collaboration.
- Enhanced understanding of software engineering concepts such as system analysis, UML diagrams, database design, documentation, and software testing.
- Developed problem-solving, debugging, and project management skills during the development process.

8.2 Future Enhancement

The StudyStore application can be further improved by adding advanced features to enhance functionality and user experience. Some possible future enhancements are:

- Integrate **online payment gateways** such as Razorpay or Stripe.
- Implement **inventory and stock management** with low-stock notifications.
- Add **product ratings and customer reviews**.
- Introduce **order cancellation and return request** features.
- Provide **real-time order tracking and email notifications**.
- Develop a **mobile application** using React Native.
- Add **AI-based product recommendations** based on user preferences.
- Generate **sales reports and analytics** for administrators.
- Implement **coupon codes and discount management**.
- Enhance application security with **Two-Factor Authentication (2FA)**.

These enhancements can improve the application's functionality, scalability, and overall user experience while making it more suitable for real-world deployment.

Chapter 9: BIBLIOGRAPHY

9.1 Online References

- React, “React Documentation,” 2026. [Online]. Available: <https://react.dev/>. [Accessed: Jul. 2026].
- Node.js, “Node.js Documentation,” 2026. [Online]. Available: <https://nodejs.org/docs/latest/api/>. [Accessed: Jul. 2026].
- Express.js, “Express.js Documentation,” 2026. [Online]. Available: <https://expressjs.com/>. [Accessed: Jul. 2026].
- MongoDB, “MongoDB Documentation,” 2026. [Online]. Available: <https://www.mongodb.com/docs/>. [Accessed: Jul. 2026].
- Mozilla Developer Network, “MDN Web Docs,” 2026. [Online]. Available: <https://developer.mozilla.org/>. [Accessed: Jul. 2026].

9.2 Offline References

- I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson Education, 2016.
- A. Banks and E. Porcello, *Learning React*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- E. Freeman, *Head First JavaScript Programming*. Sebastopol, CA, USA: O'Reilly Media, 2014.
- A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.

Thank You...